MK Praktikum Unggulan Universitas Gunadarma

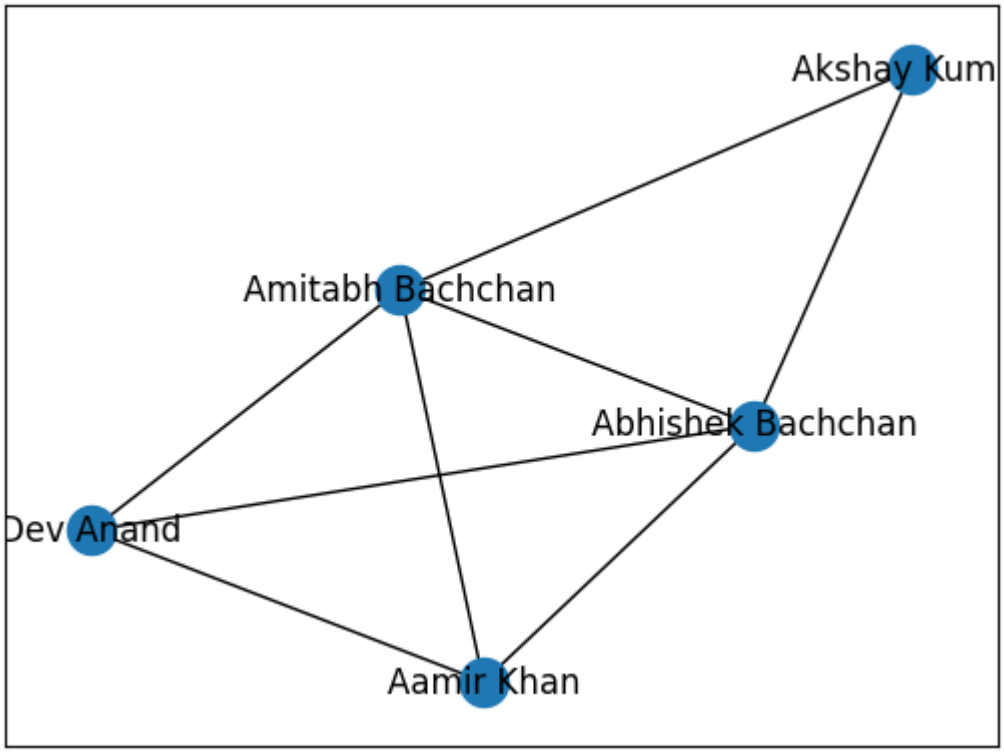
Mata Kuliah: Praktikum Teknologi Kecerdasan Artifisial (Tingkat 1)

Penjelasan Cell 1 & 2: Identitas dan Pendahuluan

Bagian ini merupakan blok Markdown yang berfungsi sebagai dokumentasi. Isinya menjelaskan bahwa ini adalah modul untuk Mata Kuliah Praktikum Teknologi Kecerdasan Artifisial di Universitas Gunadarma, dengan fokus materi pada pertemuan ke-6 yaitu Graph Search Path.

Pertemuan VI - Graph Search Path

```
import networkx as nx
G_symmetric = nx.Graph()
G_symmetric.add_edge('Amitabh Bachchan','Abhishek Bachchan')
G_symmetric.add_edge('Amitabh Bachchan','Aamir Khan')
G_symmetric.add_edge('Amitabh Bachchan','Akshay Kumar')
G_symmetric.add_edge('Amitabh Bachchan','Dev Anand')
G_symmetric.add_edge('Abhishek Bachchan','Aamir Khan')
G_symmetric.add_edge('Abhishek Bachchan','Akshay Kumar')
G_symmetric.add_edge('Abhishek Bachchan','Dev Anand')
G_symmetric.add_edge('Dev Anand','Aamir Khan')
nx.draw_networkx(G_symmetric)
G_asymmetric = nx.DiGraph()
G_asymmetric.add_edge('A','B')
G_asymmetric.add_edge('A','D')
G_asymmetric.add_edge('C','A')
G_asymmetric.add_edge('D','E')
G_asymmetric.add_edge('A','E')
```



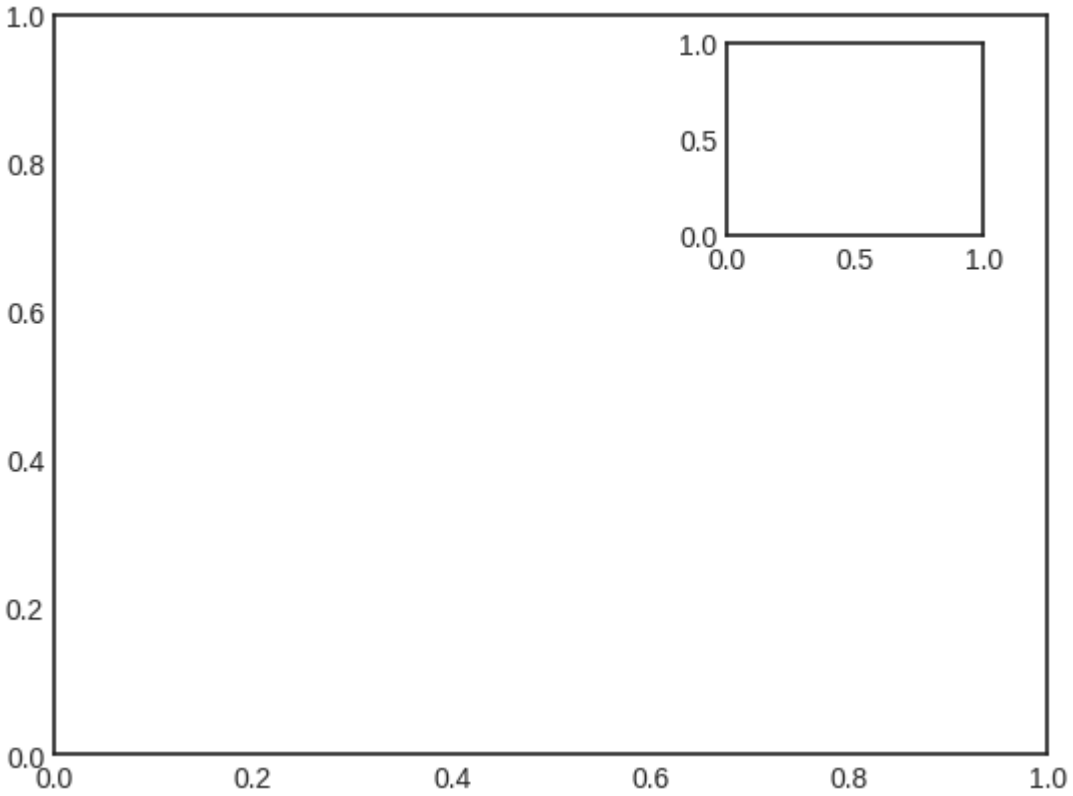
Penjelasan Cell 3: Representasi Graf dengan NetworkX

Pada sel ini, program mendemonstrasikan cara membangun dua jenis graf menggunakan pustaka networkx:

Graf Simetris (Undirected Graph): Digunakan untuk memetakan hubungan antar aktor Bollywood. Karena bersifat tidak berarah, jika A berteman dengan B, maka B otomatis berteman dengan A.

Graf Asimetris (Directed Graph/DiGraph): Menggunakan nx.DiGraph() untuk mendefinisikan hubungan satu arah (misalnya dari node A ke B), yang sering digunakan dalam pemodelan aliran data atau navigasi.

```
import matplotlib.pyplot as plt
import numpy as np

plt.style.use('seaborn-v0_8-white')
fig = plt.figure()
ax1 = plt.axes()
ax2 = plt.axes([0.65, 0.65, 0.2, 0.2])
plt.show()
```



Penjelasan Cell 4: Teknik Visualisasi Lanjut

Kode ini berfokus pada penggunaan matplotlib untuk manipulasi layout visual. Mahasiswa diperkenalkan pada cara membuat inset axes (grafik di dalam grafik) dengan menentukan koordinat kotak ([0.65, 0.65, 0.2, 0.2]).

Ini adalah teknik penting dalam penyajian data eksperimen AI untuk menunjukkan perbandingan atau detail tertentu secara simultan.

```
import networkx as nx
import numpy as np
import sys
import time

class Graph:
    def __init__(self):
        # dictionary containing keys that map to the corresponding vertex ob
        self.vertices = {}

    def add_vertex(self, key):
        """Add a vertex with the given key to the graph."""
        vertex = Vertex(key)
        self.vertices[key] = vertex

    def get_vertex(self, key):
        """Return vertex object with the corresponding key."""
        return self.vertices[key]
```

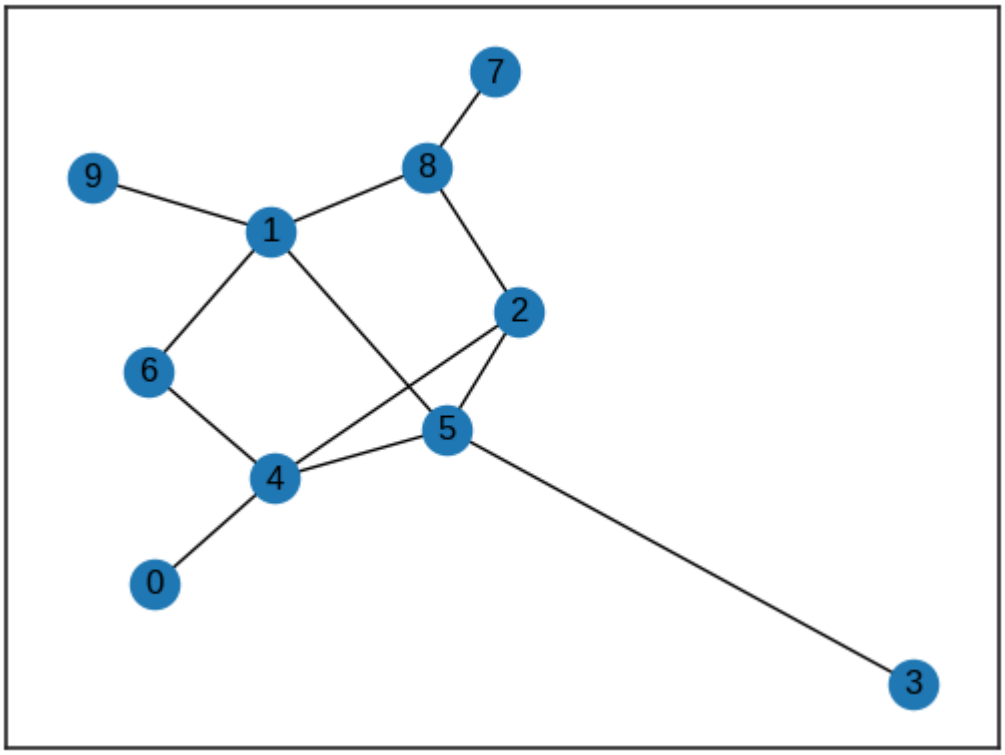
```
    def __contains__(self, key):
        return key in self.vertices

    def add_edge(self, src_key, dest_key, weight=1):
        """Add edge from src_key to dest_key with given weight."""
        self.vertices[src_key].add_neighbour(self.vertices[dest_key], weight

    def does_vertex_exist(self, key):
        return key in self.vertices

    def does_edge_exist(self, src_key, dest_key):
        """Return True if there is an edge from src_key to dest_key."""
        return self.vertices[src_key].does_it_point_to(self.vertices[dest_ke

    def display(self):
        #print('Vertices: ', end='')
        #for v in self:
        #    print(v.get_key(), end=' ')
        #print()
        list_edge = []

        #print('Edges: ')
        G_symmetric = nx.Graph()
        tot_w = 0
        tot_edge = 0
        for v in self:
            for dest in v.get_neighbours():
                w = v.get_weight(dest)
                if (int(v.get_key()) < int(dest.get_key())):
                    edge = []
                    tot_w += w
                    tot_edge += 1
                    edge.append(v.get_key())
                    edge.append(dest.get_key())
                    edge.append(w)
                    #print('(src={}, dest={}, weight={}) '.format(v.get_key(
                    #                                          dest.get_key()
                    list_edge.append(edge)
        print("Total nilai sapnning= %d dan jumlah edge= %d"%(tot_w,tot_edge
        return list_edge

    def __len__(self):
        return len(self.vertices)

    def __iter__(self):
        return iter(self.vertices.values())


class Vertex:
    def __init__(self, key):
        self.key = key
        self.points_to = {}

    def get_key(self):
        """Return key corresponding to this vertex object."""
```

```
        return self.key

    def add_neighbour(self, dest, weight):
        """Make this vertex point to dest with given edge weight."""
        self.points_to[dest] = weight

    def get_neighbours(self):
        """Return all vertices pointed to by this vertex."""
        return self.points_to.keys()

    def get_weight(self, dest):
        """Get weight of edge from this vertex to dest."""
        return self.points_to[dest]

    def does_it_point_to(self, dest):
        """Return True if this vertex points to dest."""
        return dest in self.points_to


def mst_krusal(g):
    """Return a minimum cost spanning tree of the connected graph g."""
    mst = Graph() # create new Graph object to hold the MST

    if len(g) == 1:
        u = next(iter(g)) # get the single vertex
        mst.add_vertex(u.get_key()) # add a copy of it to mst
        return mst

    # get all the edges in a list
    edges = []
    for v in g:
        for n in v.get_neighbours():
            # avoid adding two edges for each edge of the undirected graph
            if v.get_key() < n.get_key():
                edges.append((v, n))

    # sort edges
    edges.sort(key=lambda edge: edge[0].get_weight(edge[1]))

    # initially, each vertex is in its own component
    component = {}
    for i, v in enumerate(g):
        component[v] = i

    # next edge to try
    edge_index = 0

    # loop until mst has the same number of vertices as g
    while len(mst) < len(g):
        u, v = edges[edge_index]
        edge_index += 1

        # if adding edge (u, v) will not form a cycle
        if component[u] != component[v]:
```

```
                # add to mst
                if not mst.does_vertex_exist(u.get_key()):
                    mst.add_vertex(u.get_key())
                if not mst.does_vertex_exist(v.get_key()):
                    mst.add_vertex(v.get_key())
                mst.add_edge(u.get_key(), v.get_key(), u.get_weight(v))
                mst.add_edge(v.get_key(), u.get_key(), u.get_weight(v))

                # merge components of u and v
                for w in g:
                    if component[w] == component[v]:
                        component[w] = component[u]

    return mst


g = Graph()
print('Undirected Graph')
print('Menu')
print('add vertex <key>')
print('add edge <src> <dest> <weight>')
print('mst')
print('display')
print('quit')


N = 10

for i in range (0,N):
     g.add_vertex(str(i))

a = np.random.randint(2,200,(N,N), dtype=int)
print(a)

for i in range (0,N):
     for j in range (0,N):
         #if (i < j) :
            g.add_edge(str(i),str(j),a[i,j])

start = time.time()
mst = mst_krusal(g)
print('Waktu Minimum Spanning Tree:%2f'%(time.time()-start))
l_edge = mst.display()
G_sym = nx.Graph()
for e in l_edge:
    G_sym.add_edge(e[0],e[1],weight=int(e[2]))
nx.draw_networkx(G_sym)

sh_path = nx.shortest_path(G_sym,'0', str(N-1))
G_sp = nx.Graph()
print("Jalur terpendek dari node : 0 ke 9: ", sh_path)

for i in range (len(sh_path)-1):
    v1 = g.get_vertex((sh_path[i]))
    v2 = g.get_vertex((sh_path[i+1]))
```

```
        print("weight dari ",v1.get_key()," ke ",v2.get_key()," = ",v1.get_weigh
        G_sp.add_edge(v1,v2,weight=v1.get_weight(v2))

    # nx.draw_networkx(G_sp)
```

```
Undirected Graph
Menu
add vertex <key>
add edge <src> <dest> <weight>
mst
display
quit
[[157 184 187 127  43  80 103 143 151  67]
 [199 179  26 149  45  12  23 117  69  51]
 [ 61  44  55 179   7  23 141 142  24 153]
 [ 62  63 126 121 100   3 105 183 178 163]
 [110  89  17 186  61  48  30 180 104  59]
 [ 28  51 106  89 134  75  80 193  71 199]
 [190  86 117 124  93 109 103 180 168  89]
 [112 170  25 160  61 189  32  22  73 131]
 [166 117 151   3 143   3 128 196   4 139]
 [ 60  19 145  19  37 113  54 122 145  91]]
Waktu Minimum Spanning Tree:0.000435
Total nilai sapnning= 406 dan jumlah edge= 12
Jalur terpendek dari node : 0 ke 9:  ['0', '4', '5', '1', '9']
weight dari  0  ke  4  =  43
weight dari  4  ke  5  =  48
weight dari  5  ke  1  =  51
weight dari  1  ke  9  =  51
```



Penjelasan Cell 5: Implementasi Algoritma Kruskal (Core Logic)

Ini adalah bagian paling kompleks yang mencakup beberapa konsep fundamental struktur data dan algoritma:

OOP (Object-Oriented Programming): Mendefinisikan kelas Graph dan Vertex secara manual untuk merepresentasikan objek graf, bukan sekadar menggunakan library.

Algoritma Kruskal: Implementasi algoritma greedy untuk menemukan Minimum Spanning Tree (MST). Logikanya adalah mengurutkan semua edge berdasarkan bobot terkecil dan menambahkannya ke dalam MST selama tidak membentuk cycle (sirkuit).

Randomized Weighted Graph: Program menghasilkan graf dengan 10 node di mana bobot antar node ditentukan secara acak menggunakan numpy.random.randint(2, 200).

Pencarian Jalur Terpendek: Setelah MST terbentuk, program mencari jalur terpendek dari node '0' ke node '9' menggunakan fungsi nx.shortest_path. Di sini Anda bisa melihat total bobot dari MST dan detail jalur yang dilewati.

Copyright © Pengelola MK Praktikum Unggulan (Praktikum DGX), Universitas Gunadarma

https://www.praktikum-hpc.gunadarma.ac.id/